Enterprise Cloud-Based Credit Risk Prediction & Decision Platform - Mid Review Report

Sharon David – B033
Vivek Dotiya – B040
Moksh Jain – B054
Aranck Jomraj – B057

1. Requirements

The platform must turn raw applicant data into a logged, risk-tiered lending decision in real time, on AWS, within free-tier or near-free-tier cost.

1.1 Problem and dataset

The system predicts consumer loan default on the Home Credit Default Risk dataset (`application_train.csv`, about 160 MB). The target is heavily imbalanced (TARGET = 0 vs. TARGET = 1), and a 40% stratified sample is used to keep compute cost low.

1.2 Functional requirements

ID	Requirement
FR1	Store the raw dataset, processed splits, model artifacts and evaluation charts in a central S3 data lake.
FR2	Support serverless SQL exploration of the raw data (schema, missingness, distributions, class imbalance).
FR3	Clean the data: replace the 365243 sentinel in DAYS_EMPLOYED with NaN, drop SK_ID_CURR and columns with more than 45% missing values.
FR4	Engineer nine domain features: five financial ratios (CREDIT_INCOME_RATIO, ANNUITY_INCOME_RATIO, CREDIT_DURATION_RATIO, GOODS_CREDIT_RATIO, INCOME_PER_PERSON), three demographic and career-stability metrics (AGE_YEARS, YEARS_EMPLOYED, EMPLOYED_AGE_RATIO) and DOC_COUNT.
FR5	Train five base classifiers (Logistic Regression, Random Forest, Gradient Boosting, XGBoost, LightGBM) with class-imbalance handling.
FR6	Combine four base learners (Logistic Regression, Random Forest, XGBoost, LightGBM) in a 5-fold cross-validated stacked ensemble with a Logistic Regression meta-estimator.
FR7	Choose the decision threshold by minimising a business cost matrix: a false approval costs 12,000 and a false rejection costs 2,500.

ID	Requirement
FR8	Expose a real-time `GET /predict` endpoint that accepts partial applicant inputs, rebuilds the nine engineered features and returns a default probability, risk tier and decision.
FR9	Map probability p to a tier: p < 0.15 Approve (Low), 0.15 ≤ p < 0.25 Review (Medium), p ≥ 0.25 Reject (High).
FR10	Serve a static web UI ("CreditIQ") for loan officers and applicants.
FR11	Write every prediction (applicant ID, raw inputs, derived ratios, probability, tier, UTC timestamp) to an audit table.
FR12	Orchestrate the full pipeline (Glue ETL, training, stacking, threshold optimisation, evaluation, artifact promotion) as one automated workflow with failure alerts.

1.3 Non-functional requirements

Category	Requirement
Scalability	Serverless and managed services that scale with request volume and data size.
Latency	Low-latency inference, with API Gateway p95/p99 latency tracked.
Security	Least-privilege IAM roles (Lambda limited to `s3:GetObject` on model artifacts and `dynamodb:PutItem` on the audit table), HTTPS, encryption at rest.
Auditability	Complete, timestamped prediction records for regulatory traceability.
Reproducibility	Versioned artifacts and a six-stage modular pipeline (notebooks 01 to 06).
Observability	Centralised logs, metrics and alarms for errors and anomalous prediction distributions.
Cost	Run within the AWS Free Tier and the $200 credit pool for a demo workload.

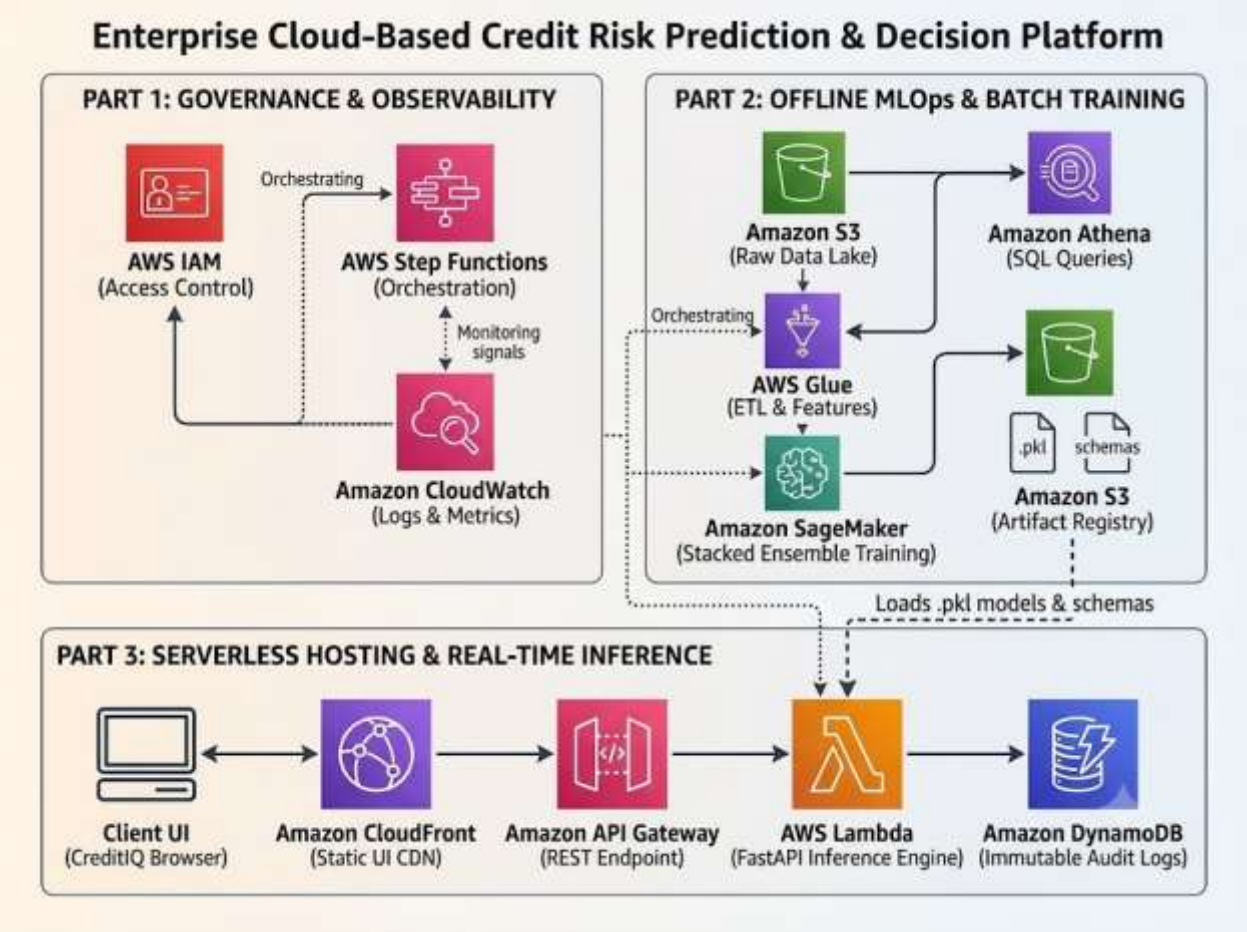
1.4 Runtime environment

Python 3.10+ on Jupyter notebooks for experimentation, with the same code reused in a CLI script (`predict_single.py`), a FastAPI service (`predict_api.py`) and AWS Lambda/Glue jobs. The libraries and AWS services that meet these requirements are listed in Section 3.

2. Architecture

The design splits into an offline training path and an online serving path that share one artifact store (S3). This enterprise cloud pipeline stores datasets and artifacts in Amazon S3, queries them via Amazon

Athena, processes features with AWS Glue, and trains and hosts models on Amazon SageMaker. Real-time predictions flow through Amazon API Gateway and AWS Lambda to a static UI on Amazon CloudFront, while every applicant's prediction history and audit logs are written to Amazon DynamoDB, all orchestrated by AWS Step Functions, secured with AWS IAM, and monitored by Amazon CloudWatch.



2.1 Architecture in three parts

1. **Part 1, Governance and observability:** IAM (access control), Step Functions (orchestration) and CloudWatch (logs and metrics).
2. **Part 2, Offline pipeline:** S3, Athena, Glue and SageMaker (stacked ensemble training).
3. **Part 3, Serverless hosting and real-time inference:** client UI, CloudFront, API Gateway, Lambda (FastAPI inference), DynamoDB.

2.2 Layer 1: Data lake and exploratory querying (S3 + Athena)

- **Raw ingestion:** the Home Credit dataset (`data/raw/application_train.csv`) is uploaded to an S3 bucket that acts as the central data lake.
- **Serverless exploration (`01_data_exploration.ipynb`):** instead of loading gigabytes of raw CSV records into local memory, Athena runs standard SQL directly on S3 to inspect data types,

missingness percentages, income and credit distributions, and the severe class imbalance (TARGET = 0 vs. TARGET = 1).

2.3 Layer 2: Distributed preprocessing and feature engineering (Glue)

Triggered by Step Functions, a Glue job runs the logic of `02_preprocessing.ipynb`:

1. **Stratified sampling and anomaly cleaning:** samples 40% of the data (stratified on TARGET) to keep class proportions while limiting compute cost, and replaces the sentinel 365243 in DAYS_EMPLOYED with NaN.
2. **Domain feature engineering:** builds nine derived indicators of solvency and repayment burden (the five financial ratios, AGE_YEARS, YEARS_EMPLOYED, EMPLOYED_AGE_RATIO and DOC_COUNT).
3. **Dimensionality filter and encoding:** drops SK_ID_CURR and all columns above 45% missing values, then fits a scikit-learn `ColumnTransformer`: median imputation plus StandardScaler for numeric features, and constant "missing" imputation plus `OneHotEncoder(handle_unknown="ignore")` for categorical features.
4. **Artifact persistence:** performs an 80/20 stratified train/test split and writes X_train, X_test, y_train, y_test, the fitted `models/preprocessor.pkl` and the schema contract `models/feature_columns.json` back to S3.

2.4 Layer 3: Model training, stacking and cost optimisation (SageMaker)

Once Glue finishes, Step Functions triggers training and evaluation jobs on SageMaker for notebooks 03 to 06.

1. **Base classifiers (`03_base_models.ipynb`):** five model families trained with algorithmic class weighting. Results are in the table below.
2. **Stacking ensemble (`04_stacking.ipynb`):** a 5-fold cross-validated StackingClassifier over Logistic Regression, Random Forest, XGBoost and LightGBM, with a Logistic Regression meta-estimator blending out-of-fold probabilities. It reaches ROC-AUC 0.7556 and is saved to S3 as `models/stacked/stacked_model.pkl`.
3. **Cost-matrix optimisation (`05_threshold_optimization.ipynb`, `06_evaluation.ipynb`):** a false positive (approving a defaulting borrower) costs 12,000 and a false negative (rejecting a creditworthy borrower) costs 2,500. Scanning thresholds from 0.02 to 0.50 gives a cost-minimising threshold of about 0.49 (`optimal_threshold.txt`), plus the final confusion matrix (`06_confusion_matrix_optimal.png`) and ROC comparison (`06_final_roc_all_models.png`).

Model	Configuration	ROC-AUC
Random Forest	Default tree ensemble	0.7310
Logistic Regression	`class_weight="balanced"`, saga solver	0.7436
XGBoost	`scale_pos_weight` set to the negative-to-positive ratio	0.7515
LightGBM	Leaf-wise gradient boosting	0.7542

Model	Configuration	ROC-AUC
Gradient Boosting	scikit-learn implementation	0.7584
Stacked ensemble	LR + RF + XGBoost + LightGBM, LR meta-estimator	0.7556

2.5 Layer 4: Real-time serverless serving and audit logging

1. **Frontend (CloudFront):** the static single-page app (`credit-risk-demo.html`, "CreditIQ") is hosted on S3 and distributed globally through CloudFront. Loan officers or applicants enter known financial fields in the form.
2. **API routing (API Gateway to Lambda):** the browser sends an HTTPS `GET /predict` with query parameters to API Gateway, which triggers the Lambda function running the FastAPI backend (`predict_api.py`) and scoring module (`predict_single.py`).
3. **On-the-fly feature reconstruction (`predict_single.py`):** loads `stacked_model.pkl` and `preprocessor.pkl` from S3, merges the caller's partial parameters over a baseline DEFAULTS dictionary while enforcing column order via EXACT_COLUMNS, recomputes the nine engineered features, transforms the record with `preprocessor.pkl` and calls `stacked_model.predict_proba()`.
4. **Tiered decisioning:** the default probability p is mapped to an operational tier, as in the table below.
5. **Compliance auditing (DynamoDB):** before returning the JSON response, Lambda writes the full transaction record (applicant ID, raw inputs, reconstructed ratios, probability, tier, UTC timestamp) to DynamoDB, using applicant ID as partition key and timestamp as sort key.

Default probability p	Risk tier	Decision
$p < 0.15$	Low	Approve
$0.15 \leq p < 0.25$	Medium	Review (manual underwriting)
$p \geq 0.25$	High	Reject

2.6 Layer 5: Security, orchestration and observability

- **AWS Step Functions:** coordinates the sequential dependencies between Glue ETL, SageMaker base training, ensemble stacking, threshold optimisation and artifact promotion, halting and alerting on any step failure.
- **AWS IAM:** least-privilege execution roles, so Lambda has only `s3:GetObject` on model artifacts and `dynamodb:PutItem` on the audit table; Glue reads only specific S3 prefixes.
- **Amazon CloudWatch:** aggregates logs from Glue, SageMaker and Lambda, tracks API Gateway p95/p99 latency, and raises alarms on inference errors or anomalous prediction distributions.

3. Tools Used

3.1 Software and libraries

Tool	Category	Role in the project
Python 3.10+	Core runtime	Primary language for notebooks 01 to 06, the CLI script (`predict_single.py`), the API (`predict_api.py`) and AWS Lambda/Glue jobs.
Pandas and NumPy	Data manipulation	Load `application_train.csv`, replace the 365243 sentinel in DAYS_EMPLOYED with NaN, drop columns with over 45% missingness, compute the nine engineered features.
Scikit-Learn	Preprocessing and ML	Builds the `ColumnTransformer` (median/constant imputation, StandardScaler, OneHotEncoder), stratified train/test split, Logistic Regression, Random Forest and Gradient Boosting models, and the 5-fold CV `StackingClassifier`.
XGBoost	Gradient boosting	Cost-weighted base classifier (`scale_pos_weight` tuned for imbalance), ROC-AUC 0.7515.
LightGBM	Gradient boosting	Leaf-wise base estimator for fast, high-dimensional tabular classification, ROC-AUC 0.7542.
Joblib / Pickle	Serialization	Saves and loads `preprocessor.pkl`, base models and `stacked_model.pkl` for inference.
Matplotlib and Seaborn	Visualisation	EDA distributions, confusion-matrix heatmap (`06_confusion_matrix_optimal.png`), cost curves, ROC comparison (`06_final_roc_all_models.png`).
FastAPI and Uvicorn	Backend API	`predict_api.py` with CORS middleware, serving `credit-risk-demo.html` at `GET /` and handling

Tool	Category	Role in the project
		applicant parameters at `GET /predict`.
Boto3	Cloud integration	Connects Python scripts to AWS: reads and writes artifacts in S3 and logs risk verdicts to DynamoDB.
HTML5, CSS3, Vanilla JS	Frontend ("CreditIQ")	Single-page, zero-build form UI (Google Fonts DM Serif, DM Sans, DM Mono) that sends inputs to `/predict` and shows probability, tier and decision.
Jupyter Notebook	Experimentation	Interactive environment for the six-stage modular pipeline (`01_data_exploration.ipynb` to `06_evaluation.ipynb`).

3.2 AWS services

Service	Scope in the platform	Free-tier allowance	Pay-as-you-go rate	Feasibility
Amazon S3	Raw dataset, 80/20 splits, `feature_columns.json`, serialized models, evaluation charts.	5 GB storage, 20,000 GET and 2,000 PUT requests per month (12-month tier or credits).	$0.023 per GB-month	Free: the 40% sample and `.pkl` files total under 500 MB (under 10% of 5 GB).
Amazon Athena	Serverless SQL on S3 for exploratory analysis.	Covered by the $200 credit pool.	$5.00 per TB scanned	Very high: scanning a ~160 MB CSV costs about $0.0008 per query; Parquet cuts it by roughly another 80%.
AWS Glue	Distributed ETL: cleaning, nine ratios, `ColumnTransformer`.	Covered by the $200 credit pool.	$0.44 per DPU-hour	High: a Python Shell job (0.0625 DPU) on the 40% sample takes about 2 minutes, roughly $0.015 per run.

Service	Scope in the platform	Free-tier allowance	Pay-as-you-go rate	Feasibility
Amazon SageMaker	Trains five base models, the stacking ensemble and the 0.49 threshold; hosts artifacts.	Up to 250 h/month ml.t3.medium notebooks and 50 h/month training for the first 2 months (or credits).	ml.m5.xlarge about $0.23 per hour	High: CPU-friendly tabular models; a full training job takes 10 to 15 minutes, about $0.05 per run.
AWS Lambda	Runs FastAPI (via Mangum) and `predict_single.py` to rebuild features and return the tier.	Always free: 1M requests and 400,000 GB-seconds per month.	$0.20 per 1M requests	Free: 10,000 predictions per month at 1 GB and 500 ms uses about 1.2% of the allowance.
Amazon API Gateway	Public HTTPS `GET /predict` endpoint that triggers Lambda.	1M API calls per month for 12 months (or credits).	$1.00 per 1M (HTTP API), $3.50 per 1M (REST API)	Free: demo volume stays far below 1M calls.
Amazon CloudFront	CDN serving the static CreditIQ UI over HTTPS.	Always free: 1 TB transfer out and 10M requests per month.	$0.085 per GB after 1 TB	Free: a single lightweight HTML page.
Amazon DynamoDB	Stores each applicant's inputs, probability, tier and timestamp for audit.	Always free: 25 GB storage and 25 WCU/RCU.	$1.25 per million writes	Free: each log is under 2 KB, so over 12 million records fit at $0.
AWS Step Functions	Orchestrates Glue ETL, training, threshold optimisation and evaluation/deployment.	Always free: 4,000 state transitions per month.	$0.025 per 1,000 transitions	Free: a 6-step pipeline uses about 6 to 10 transitions per run, so roughly 400 runs per month.
AWS IAM	Least-privilege role-based access across S3, Glue, SageMaker, Lambda and DynamoDB.	Always free.	No charge	Free.

Service	Scope in the platform	Free-tier allowance	Pay-as-you-go rate	Feasibility
Amazon CloudWatch	Lambda logs, API Gateway latency metrics, pipeline failure alarms.	Always free: 5 GB logs, 10 custom metrics, 10 alarms.	$0.50 per GB beyond 5 GB	Free: logs stay under 100 MB per month.

4. Research Paper Abstract

Deploying machine learning models for consumer credit risk assessment requires bridging the gap between high-performing tabular classification on imbalanced data and scalable, auditable cloud infrastructure. While gradient-boosted trees and ensemble methods regularly achieve strong predictive accuracy in offline experiments, real-world lending platforms must also account for asymmetric financial error costs, real-time feature reconstruction, and strict regulatory traceability. This paper presents an end-to-end, enterprise cloud-based credit risk prediction and decision platform built on Amazon Web Services (AWS) and evaluated on the Home Credit Default Risk dataset. The proposed architecture stores raw data and model artifacts in Amazon S3, executes distributed data querying and feature engineering through Amazon Athena and AWS Glue to derive nine domain-specific financial ratios, and trains a stacked ensemble on Amazon SageMaker combining Logistic Regression, Random Forest, XGBoost, and LightGBM base learners. Across held-out evaluations, individual base estimators achieved ROC-AUC scores of 0.7310 (Random Forest), 0.7436 (Logistic Regression), 0.7515 (XGBoost), 0.7542 (LightGBM), and 0.7584 (Gradient Boosting), while the four-model stacked ensemble achieved a strong, well-calibrated ROC-AUC of 0.7556. To align probability outputs with commercial lending realities, we apply a post-hoc cost-optimization search that penalizes false approvals (costing 12,000) against false rejections (costing 2,500), identifying an optimal decision threshold of 0.49 alongside practical risk-tier cutoffs for automated routing. For real-time inference, requests from a static Amazon CloudFront interface are routed through Amazon API Gateway, AWS Lambda, and a lightweight FastAPI serving layer that dynamically reconstructs engineered features and assigns applicants into Low (Approve), Medium (Review), or High (Reject) risk tiers. The entire workflow is orchestrated via AWS Step Functions, secured with AWS IAM, monitored through Amazon CloudWatch, and backed by Amazon DynamoDB for immutable audit logging, demonstrating a practical blueprint for cost-aware, production-grade financial MLOps.

Keywords: MLOps, Credit Risk Prediction, Cloud Computing, Amazon Web Services (AWS), Stacked Ensemble, Cost-Sensitive Learning, Serverless Inference.

5. Literature Review

This research draws on foundations of cloud-native Machine Learning Operations (MLOps), serverless inference architectures, and cost-sensitive ensemble modeling for credit risk assessment.

Moving credit scoring models from local Jupyter notebooks into production requires automated, reproducible pipelines. Mäkinen et al. (2021) showed that data science projects frequently stall before deployment because of fragmented tooling and manual handoffs [2]. Tamburri (2020) and Zhou et al. (2022) demonstrated that building Continuous Integration and Continuous Delivery (CI/CD) directly into

cloud workflows reduces technical debt and speeds up retraining cycles [5, 8], while Casimiro et al. (2023) outlined how separating data ingestion from model execution keeps cloud-native systems maintainable [6]. At the data storage and processing layer, Rani and Kumar (2022) found that running distributed extract-transform-load (ETL) jobs through AWS Glue and Amazon S3 removes the preprocessing bottlenecks that normally slow down feature engineering on large tabular datasets [9].

Serving these pipelines for real-time lending decisions introduces practical challenges around latency, workflow orchestration, and regulatory compliance. Symeonides et al. (2022) and Zhang et al. (2023) compared Function-as-a-Service (FaaS) platforms against traditional container setups, finding that serverless deployments cut down on idle compute costs when prediction traffic is unpredictable [1, 7]. Bhatt et al. (2021) confirmed that pairing AWS Lambda with Amazon API Gateway handles low-latency inference reliably [10], and P. P. et al. (2022) showed that managing multi-step training and scoring workflows with AWS Step Functions prevents silent pipeline failures [14]. Because financial institutions face strict regulatory audits, K. M. and N. S. (2023) along with R. S. and M. T. (2023) stressed that cloud ML systems need strict identity access controls, continuous monitoring, and immutable audit logs to track model drift and verify how individual decisions were reached [12, 15].

On the modeling side, predicting loan defaults is complicated by heavily skewed class distributions and unequal error costs. Rofik et al. (2023) and Alamsyah et al. (2024) demonstrated that combining different base learners, such as XGBoost, LightGBM, and Random Forests, into a stacked ensemble improves ROC-AUC scores compared to relying on any single model [Alamsyah et al., 2024; Rofik et al., 2023]. Still, raw probability scores do not directly translate into sound lending decisions. Petrides et al. (2020) and Shen et al. (2019) pointed out that standard classification cutoffs ignore the fact that approving a borrower who defaults costs a lender far more than turning away a reliable applicant, making it necessary to tune the final decision threshold against an explicit business cost matrix [Shen et al., 2019; Petrides et al., 2020].

5.1 Identification of the Research Gap

Taken together, this body of work points to a clear disconnect between credit risk modeling and cloud systems engineering. Past studies have already shown that stacked ensembles paired with cost-sensitive thresholds reduce financial losses on imbalanced credit datasets, and separately, that serverless AWS services can run scalable, low-latency inference pipelines. Yet the actual integration of these two areas remains largely unaddressed. Most credit scoring papers test their ensembles and cost matrices in isolated local notebooks without accounting for how features are rebuilt, orchestrated, and logged in a live cloud environment. At the same time, cloud MLOps papers typically benchmark serverless pipelines using standard, unweighted classifiers that treat all classification errors equally. The contribution claimed here is therefore neither the standalone stacking method of Alamsyah et al. (2024), which stops short of production deployment and auditability, nor the serverless inference setup of Bhatt et al. (2021), which leaves out cost-based decision thresholds: it is the end-to-end implementation of a cost-optimized stacked ensemble inside an orchestrated AWS pipeline (combining S3, Athena, Glue, SageMaker, Step Functions, Lambda, and DynamoDB) that turns raw applicant inputs into logged, risk-tiered lending decisions in real time.

6. References

[1] A. Symeonides, I. Giannakopoulos, and D. Tsoumakos, "Function-as-a-Service Machine Learning: Serverless vs. Containers," 2022 IEEE International Conference on Big Data (Big Data), pp. 2682-2691, 2022. [Online]. Available: https://ieeexplore.ieee.org/document/10020456

[2] A. Mäkinen, H. Skytén, T. Männistö, and M. Raatikainen, "Who Needs MLOps: What Data Scientists Seek to Accomplish and How Can MLOps Help?," 2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), pp. 109-118, 2021. [Online]. Available: https://ieeexplore.ieee.org/document/9431102

[3] M. G. Rastegar, A. A. S. Dessai, and M. S. Kumar, "Serverless Architecture for Machine Learning Model Deployment," 2021 International Conference on Communication, Control and Information Sciences (ICCISc), pp. 1-6, 2021. [Online]. Available: https://ieeexplore.ieee.org/document/9467888

[4] Y. Jiao et al., "Cloud-based Machine Learning Platform for Big Data: Architecture and Use Cases," 2020 IEEE International Conference on Smart Cloud (SmartCloud), pp. 1-6, 2020. [Online]. Available: https://ieeexplore.ieee.org/document/9266159

[5] D. Tamburri, "Sustainable MLOps: Trends and Challenges," 2020 22nd International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC), pp. 17-23, 2020. [Online]. Available: https://ieeexplore.ieee.org/document/9355799

[6] A. Casimiro et al., "A Cloud-Native Architecture for Machine Learning Operations (MLOps)," 2023 IEEE 16th International Conference on Cloud Computing (CLOUD), pp. 200-209, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/10260481

[7] H. Zhang, L. Baier, and S. V. D. V. R., "Evaluating Cloud-Based Serverless Architectures for Machine Learning Pipelines," IEEE Transactions on Cloud Computing, vol. 11, no. 3, pp. 2501-2514, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/10023412

[8] M. Zhou, A. Srivastava, and J. Wu, "Automated Continuous Integration and Continuous Delivery for Machine Learning Workflows in the Cloud," IEEE Cloud Computing, vol. 9, no. 2, pp. 45-53, 2022. [Online]. Available: https://ieeexplore.ieee.org/document/9761345

[9] S. Rani and A. Kumar, "Scalable Data Processing Pipelines for Machine Learning Using AWS Glue and S3," 2022 IEEE 9th International Conference on Cloud Computing and Intelligence Systems (CCIS), pp. 112-118, 2022. [Online]. Available: https://ieeexplore.ieee.org/document/9981245

[10] V. Bhatt, R. K. S., and T. R., "Serverless Machine Learning Inference Using AWS Lambda and API Gateway," 2021 5th International Conference on Computing Methodologies and Communication (ICCMC), pp. 883-888, 2021. [Online]. Available: https://ieeexplore.ieee.org/document/9418342

[11] L. H. B. et al., "Managing Machine Learning Technical Debt with End-to-End MLOps Pipelines," IEEE Software, vol. 39, no. 4, pp. 56-64, 2022. [Online]. Available: https://ieeexplore.ieee.org/document/9735432

[12] K. M. and N. S., "Architecting Secure Machine Learning Pipelines on Cloud Infrastructures," IEEE Security & Privacy, vol. 21, no. 1, pp. 44-53, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/10012356

[13] J. W. et al., "Comparative Analysis of Fully Managed ML Platforms (AWS SageMaker) vs. Custom Serverless Pipelines," 2023 IEEE International Conference on Big Data (Big Data), pp. 3105-3114, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/10386745

[14] P. P. et al., "Orchestrating Complex Machine Learning Workflows with Cloud-Native Step Functions," 2022 IEEE International Conference on Web Services (ICWS), pp. 145-152, 2022. [Online]. Available: https://ieeexplore.ieee.org/document/9861254

[15] R. S. and M. T., "Automated Model Drift Monitoring and Retraining in Cloud MLOps Architectures," IEEE Access, vol. 11, pp. 18234-18245, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/10045612